

Teaching a Small Hebrew Model to Explain Bureaucracy:

Fine-Tuning DictaLM 1.7B with Teacher-Generated Data

Ahmad Tawil

ahmadtawil.se@gmail.com

Code and data: <https://github.com/AhmadTawil1/bureaucracy-navigator>

Model: <https://huggingface.co/AhmadTawil1/dictalm3-bureaucracy-lora>

September 2026

Abstract

Israeli bureaucracy is hard to follow, and the best free guide to it, Kol Zchut, is long and written in formal Hebrew. I asked whether a *small* open Hebrew model (1.7 billion parameters, small enough to run on a laptop) can be taught to answer questions about it in simple Hebrew, cite its sources, say “I don’t know” when the sources don’t cover a question, and summarize official letters in a fixed format. I built a training set of 1,128 examples from 287 Kol Zchut articles using a larger model (12B) as a “teacher”, fine-tuned DictaLM 3.0 1.7B with LoRA in about 20 minutes, and compared the base model, the fine-tuned model and the teacher on 232 test examples from articles never seen in training. Fine-tuning taught behaviors the base model did not have at all: refusing when the sources don’t answer (0% $\rightarrow$ 61%), following the letter format (28% $\rightarrow$ 100%) and writing plain text (49% $\rightarrow$ 100%). In a manual check of 30 questions, correct answers doubled (8 $\rightarrow$ 16) and fully grounded answers rose from 15 to 26, while answers also got 36% faster on a laptop CPU. The main cost is that the model now refuses 12% of questions it could have answered. Most of the useful lessons came from reading the data: several steps that looked fine in the plan produced bad training examples until they were checked by hand.

1 Introduction

People who lose a job, ask for a tax refund or receive a letter from the National Insurance Institute (Bituah Leumi) often don’t know what they are entitled to, what they must do, or by when. Kol Zchut (<https://www.kolzchut.org.il>) collects this information in thousands of articles, but a user still has to find the right article and read it.

A chatbot built on a large commercial model could help, but it sends personal documents to an outside service and costs money per message. I wanted to know how far a *small, local, open* model can go instead. Small models have typical weaknesses for this job: they ignore the context they are given, forget to cite sources, answer even when they don’t know, and don’t follow output formats. These are exactly the kinds of behaviors that fine-tuning can change, and they can all be measured automatically.

The resulting system is a Telegram bot. A user either asks a question, or sends a PDF or a photo of an official letter. The bot finds relevant Kol Zchut passages, and the fine-tuned model writes the answer or the summary.

2 Data

Knowledge base. I downloaded articles from four Kol Zchut categories (unemployment, income support, income tax, dismissal) through the site’s public API. Kol Zchut labels each article with a

type; I kept only practical ones (rights, procedures, services, guides) and skipped court rulings, index pages and short definitions, which made up about 20% of the articles and don’t answer “what am I entitled to and how do I get it”. That left **287 articles**.

I removed page clutter (menus, edit links, contact tables) and split each article into passages of up to 384 tokens, the unit my search model reads. A first attempt that split every section separately produced 707 tiny passages of under 30 words, which match searches by chance and give the model too little to work with. Packing neighboring sections of the same article together gave **1,688 passages** averaging 150 words, each keeping its section headings. The passages were embedded with a multilingual search model (e5-large) and stored in a vector database (Qdrant).

Training examples. I used DictaLM 3.0 12B, a larger model from the same family, as a teacher to write examples for four tasks (Table 1). The questions themselves were also written by the teacher (three realistic citizen questions per passage), then sampled evenly across articles.

Task	What the model sees	What it should write
Grounded questions and answers	a question and 4 retrieved passages, in random order	a short answer that cites passages as [1], [2], ...
Unanswerable questions	a question and 4 passages that look related but don’t answer it	a fixed refusal: “I didn’t find reliable information about this in my sources”
Letter summaries	an invented official letter with fake personal details, plus 3 passages	a summary under four headings: what the letter says, what to do, by when, whom to contact
Plain-Hebrew rewrites	a long bureaucratic paragraph	the same meaning in short, simple sentences

Table 1: The four training tasks.

Quality control. Automatic filters removed examples with citations to passages that don’t exist, answers with no citation, summaries missing a heading or the letter’s deadline, very long answers, non-Hebrew text, leftover placeholders and near-duplicate questions. Then I read 50 random examples by hand, which led to the most important fixes (Section 6). In the end **1,628 of 1,776** generated examples were kept.

Split by article. I held out 15% of the articles (40 of 269) for testing, so test questions are about topics the model never saw. I went one step further than the plan: training examples whose retrieved passages included text from a test article were also removed, so no test text reaches the model during training. Final sizes: **1,128 training, 125 validation and 232 test examples**.

3 Model and training

The student model is dicta-il/DictaLM-3.0-1.7B-Instruct. I fine-tuned it with **LoRA**, which trains small add-on weight matrices instead of the whole model, using Unsloth and TRL on a single Colab L4 GPU (rank 16, alpha 32, all attention and feed-forward projections). The loss was computed on the model’s answers only, not on the long prompts. Training ran for 2 epochs (142 steps) and took about **20 minutes**. The validation loss was still falling at the end (best at step 140 of 142), so the model did not overfit. The prompts are shared code between data generation and the bot, so the model sees exactly the same format in use as in training.

For use on a laptop, the adapters were merged into the model and exported to GGUF (8-bit, 1.8 GB), which runs with llama.cpp. The base model was converted the same way so both could be compared fairly.

4 Evaluation

I compared three models on the same 232 test examples, with the same retrieved passages for each: the **base** 1.7B model, my **fine-tuned** 1.7B model, and the **12B teacher** as an upper reference. All answered with temperature 0.

Automatic metrics check behavior rather than style: whether answers cite only real passages, whether the model refuses exactly when it should, whether letter summaries have all four headings and the right deadline, whether the output is plain text, sentence length, and seconds per answer on a laptop CPU.

Manual scoring. For 30 test questions, the base and fine-tuned answers were shuffled into one file with the model name hidden, and each answer was marked correct, partial or wrong, and grounded or not (every claim supported by the passages). This scoring was done with an AI assistant rather than a human expert, and the two models are recognizable by style, so it is not a fully blind human evaluation.

End-to-end letters. Five invented letters (Bituah Leumi, the tax authority, an employer, a landlord, a municipality) were each tested as a PDF made in Word and as a phone photo, through the bot’s full pipeline: text extraction, masking of personal details, search and summarization.

5 Results

Metric	Base 1.7B	Fine-tuned 1.7B	Teacher 12B
Answers with valid citations	81%	84%	100%
Refuses when the passages don’t answer ($n = 18$)	0%	61%	0%
Refuses when it could have answered ($n = 116$)	0%	12%	0%
Letter summaries with all 4 headings ($n = 40$)	28%	100%	100%
Plain text, no markdown	49%	100%	84%
Correct answers, manual (of 30)	8	16	–
Fully grounded answers, manual (of 30)	15	26	–
Seconds per answer, laptop CPU	51.1	32.6	–

Table 2: Results on 232 held-out test examples (articles never seen in training).

What changed. The clearest effect is on behaviors the base model simply didn’t have (Table 2). It never refused: when the passages didn’t contain the answer, it answered anyway from the nearest passage. The fine-tuned model refuses in 61% of those cases. It also follows the letter format every time and never uses markdown, which matters because Telegram shows markdown as raw symbols. The teacher, although much larger, *also never refused*; refusal came from my unanswerable examples, not from copying the teacher.

In the manual check, the base model’s typical answer was “partial”: the right fact buried in a long markdown dump, sometimes repeating itself in a loop. The fine-tuned model’s answers were shorter and twice as often fully correct, which also explains why they are faster to generate.

What it costs. The fine-tuned model refuses 12% of questions it could have answered; 4 of its 7 wrong answers in the manual check were such refusals. On some questions whose passages are long step-by-step lists, it copies the list instead of answering, without citations.

Letters. Text came out in the correct reading order in all 10 runs (PDFs: 100% of words recovered; photos: 92–99%). All 10 summaries had the four headings and masked the ID number. The deadline under “by when” was right in 7 of 10: one photo’s date was misread by OCR, and for the landlord letter the model put the deposit terms there instead of the move-out date.

6 What I learned

Most improvements came from looking at intermediate outputs, not from the original plan:

- **A tokenizer bug made the teacher answer nonsense.** The teacher’s configuration named a tokenizer type that the current version of the transformers library rebuilds incorrectly, *deleting all Hebrew*: a whole question became a single “?”, and the model answered about a music album. Loading the same tokenizer file through a generic class fixed it. A quick look at the input token count (80 instead of 1,300) would have caught this immediately.
- **“Other articles” are not automatically unrelated.** The planned way to build unanswerable examples was to pair a question with passages from different articles. Kol Zchut articles overlap so much that those passages often did answer the question, which would have trained the model to refuse answerable questions. I had the teacher judge each candidate and kept only real negatives: just 148 of 1,500 passed.
- **Sampling temperature matters for facts.** The first answers were generated at temperature 0.7, copied from the plan. A manual review found wrong or invented facts in about a third of them. Regenerating at 0.2 brought this down to roughly 10–15%.
- **A model checking its own work caught almost nothing.** Asking the teacher to verify its answers marked 822 of 850 as fine and caught only 1 of 8 answers I knew were wrong. Reading samples by hand was far more useful.
- **Automatic checks can be too kind.** My first letter test passed 10 of 10 on deadlines because it only checked that the date appeared somewhere in the summary. Reading the summaries showed three were wrong; requiring the date under the “by when” heading gave the honest 7 of 10.
- **Real documents break simple patterns.** In PDFs exported from Word, numbers were glued to the Hebrew word next to them. My masking rule looked for a word boundary before the number, and there is none between a Hebrew letter and a digit, so the *ID and phone number were not masked*. I only found this by testing a real PDF.

7 Limitations

- The training data was written by a teacher model and carries some of its mistakes, even after filtering and review.
- The test set is small for some measures (18 unanswerable questions, 9 letters with a checkable deadline), and the manual scoring was done by an AI assistant, not a human expert.
- The model over-refuses (12%) and sometimes copies long passages instead of answering.
- It covers only four topic areas and is only as current as the downloaded articles.
- Masking can’t tell the recipient’s phone number from an office contact number, so it hides both.
- The bot gives general information, not legal advice.

8 Conclusion

A 1.7B model, fine-tuned for 20 minutes on about a thousand teacher-generated examples, learned behaviors that matter for a trustworthy assistant and that its base version lacked completely: refusing when it doesn't know, citing sources, and following a fixed summary format. It doubled the share of correct answers and runs faster on an ordinary laptop, with no personal document leaving the machine except through Telegram. The trade-off is over-caution, and the obvious next steps are more answerable examples that look like hard cases, and a larger human-scored test set. The broader lesson is practical: with synthetic data, most of the quality comes from reading the data at every step.

Acknowledgments and license

Content from Kol Zchut (<https://www.kolzchut.org.il>), used under CC BY-NC-SA 2.5 IL; all derived data and the adapters are shared under the same non-commercial terms. Models by Dicta (<https://huggingface.co/dicta-il>). Built with the help of Claude (Anthropic) as a coding assistant.